
Software Requirements Specification

for

GOVERNMENT SMART SCHOOL GRIEVANCE PORTAL

Version 1.0 Approved

Prepared by Lavish Sharma

Department of Computer Science and Engineering

Coimbatore Institute Of Technology

15thOctober2026

Table of Contents

1. 1. Introduction	2
o 1.1 Purpose	2
o 1.2 Document Conventions	3
o 1.3 Intended Audience and Reading Suggestions	3
o 1.4 Product Scope	3
o 1.5 References	4
2. 2. Overall Description	4
o 2.1 Product Perspective	4
o 2.2 Product Functions	4
o 2.3 User Classes and Characteristics	5
o 2.4 Operating Environment	16
o 2.5 Design and Implementation Constraints	16
o 2.6 User Documentation	16
o 2.7 Assumptions and Dependencies	16
3. 3. External Interface Requirements	17
o 3.1 User Interfaces	17
o 3.2 Hardware Interfaces	18
o 3.3 Software Interfaces	18
o 3.4 Communication Interfaces	18
4. 4. System Features	19
o 4.1 User Authentication and Authorization	19
o 4.2 Grievance Management	20
o 4.3 Complaint Tracking	21
o 4.4 Report Management	22
o 5. Other Nonfunctional Requirements	22
5. 5.1 Performance Requirements	22
o 5.2 Safety Requirements	22
o 5.3 Security Requirements	23
o 5.4 Software Quality Attributes	23
o 5.5 Business Rules	23
o 6. Other Requirements	24

6. 6.1 Database Requirements	24
6.2 Legal Requirements	24
Appendix: Analysis Models	25
Revision History	2
Revision History	2

Revision History

Name	Date	ReasonForChanges	Version

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for the Government Smart School Grievance Portal.

The system provides an online platform where students, parents, teachers, school staff, and authorized government officials can submit, manage, monitor, and resolve grievances related to government schools.

The portal is designed to improve transparency, accountability, communication, and efficiency in grievance handling. It replaces traditional manual complaint registration and tracking with a centralized digital system.

1.2 Document Conventions

- SRS refers to Software Requirements Specification.
- Admin refers to an authorized School Administrator.
- FR refers to Functional Requirement.
- NFR refers to Non-Functional Requirement.
- REQ refers to a specific system requirement.
- High Priority requirements are mandatory requirements.
- Shall indicates a mandatory requirement.
- Should indicates a recommended requirement.
- May indicates an optional requirement.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Project Developers
- System Analysts
- Project Managers
- Database Administrators
- System Test Engineers
- School Administrators

- Teachers
- Government Education Officials

Readers should begin with the Introduction and Overall Description sections before proceeding to External Interface Requirements, System Features, Nonfunctional Requirements, and Analysis Models.

1.4 Product Scope

The Government Smart School Grievance Portal is a web-based application designed to provide a centralized platform for registering, processing, tracking, and resolving grievances related to government schools.

The system allows:

- Students to submit academic and school-related grievances.
- Parents to register complaints related to students and school facilities.
- Teachers to submit school-related grievances.
- School Administrators to review, assign, update, and resolve complaints.
- Government Officials to monitor grievance statistics and generate reports.
- Users to track the progress of complaints using unique complaint IDs.
- Administrators to maintain grievance records and status history.
- The system to generate notifications and analytical reports.

The system improves grievance management by providing digital registration, transparent status tracking, centralized records, and reporting facilities.

1.5 References

1. IEEE Recommended Practice for Software Requirements Specifications.
2. Software Engineering reference materials.
3. UML Modeling Standards.
4. HTML, CSS and JavaScript Documentation.
5. MySQL Database Documentation.
6. Government Education Department Guidelines.

2. Overall Description

2.1 Product Perspective

The Government Smart School Grievance Portal is a standalone web-based application designed for government educational institutions. It replaces traditional manual complaint handling with a centralized digital platform for grievance registration, assignment, tracking, resolution, and reporting.

2.2 Product Functions

- User Registration and Authentication
- Role-Based Access Control
- Grievance Submission
- Complaint Categorization
- Complaint ID Generation
- Complaint Assignment
- Complaint Tracking
- Status Updates
- Document Upload
- Notifications
- Report Generation
- Dashboard Analytics
- Grievance History Management

2.3 User Classes and Characteristics

Student Use Case Description

UC-01: Submit Grievance

Primary Actor: Student

Secondary Actor: Database Server

Trigger: Student selects the "Submit Grievance" option.

Preconditions:

- Student is registered.
- Student is successfully logged in.

Postconditions:

- Grievance is stored in the database.
- A unique complaint ID is generated.
- The grievance is assigned the appropriate status.

Main Success Scenario:

1. Student logs into the portal.
2. Student selects "Submit Grievance".
3. System displays the grievance form.
4. Student selects a complaint category.
5. Student enters grievance details.
6. Student uploads supporting documents if required.
7. Student submits the grievance.
8. System validates the information.
9. System generates a unique complaint ID.
10. System stores the grievance.
11. System displays a submission confirmation.

Exceptions:

1. Required information missing: System asks the student to complete mandatory fields.
2. Invalid document: System rejects the unsupported file.
3. Session timeout: System redirects the student to the login page.

UC-02: Track Grievance

Primary Actor: Student

Secondary Actor: Database Server

Trigger: Student enters a complaint ID in the tracking interface.

Preconditions:

- Student is logged in.
- A valid complaint ID exists.

Postconditions:

- Current grievance status is displayed.

Main Success Scenario:

1. Student opens the grievance tracking page.
2. Student enters the complaint ID.
3. System validates the complaint ID.

4. System retrieves the grievance record.
5. System displays the current status.
6. System displays the grievance history and previous updates.

Exceptions:

1. Invalid complaint ID: System displays an appropriate error message.
2. Record not found: System informs the user that the grievance could not be found.

UC-03: Receive Grievance Notification

Primary Actor: Student

Secondary Actor: Notification Service

Trigger: Administrator updates grievance status.

Precondition:

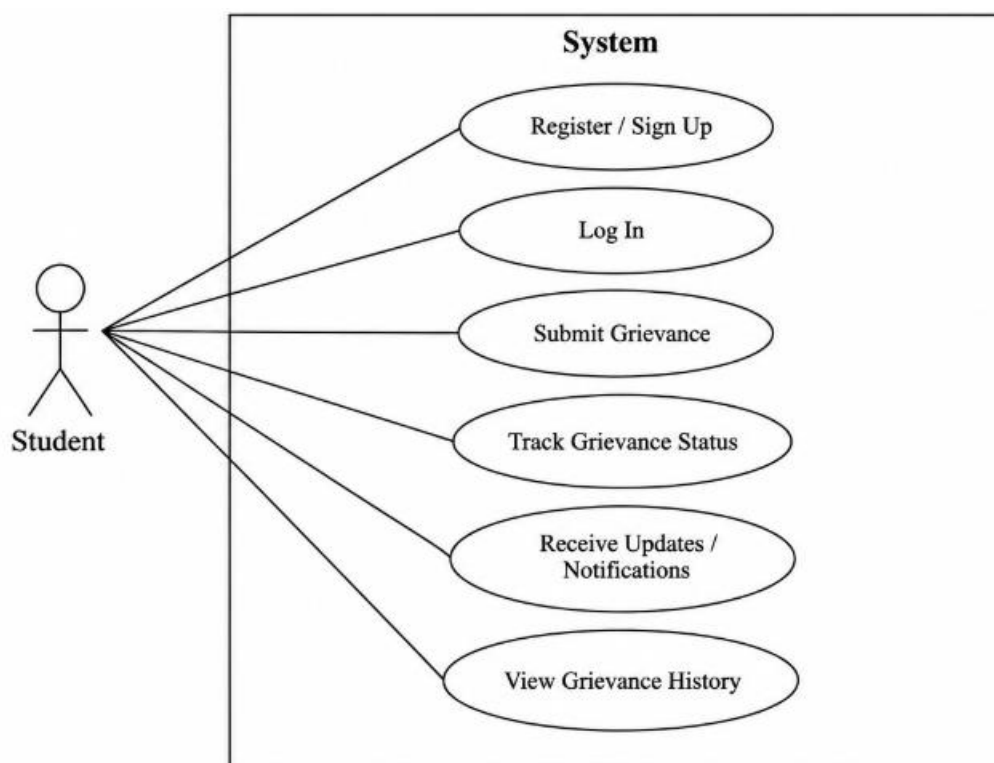
- Student has an active grievance.

Postcondition:

- Student receives a notification about the status change.

Main Success Scenario:

1. Administrator updates grievance status.
2. System records the status change.
3. System identifies the concerned user.
4. Notification service generates an alert.
5. Student receives the notification.
6. Updated grievance status becomes visible in the dashboard.



Parent Use Case Description

UC-04: Register Parent Grievance

Primary Actor: Parent

Secondary Actor: Database Server

Trigger: Parent selects the grievance registration option.

Preconditions:

- Parent has a valid account.
- Parent is authenticated.

Postconditions:

- Complaint is registered successfully.

Main Success Scenario:

1. Parent logs into the portal.
2. Parent selects "New Grievance".
3. Parent selects the appropriate category.

4. Parent enters the complaint description.
5. Parent provides student or school information.
6. Parent uploads supporting documents if necessary.
7. Parent submits the grievance.
8. System validates the information.
9. System generates a unique complaint ID.
10. System stores the grievance.
11. System confirms successful submission.

Exceptions:

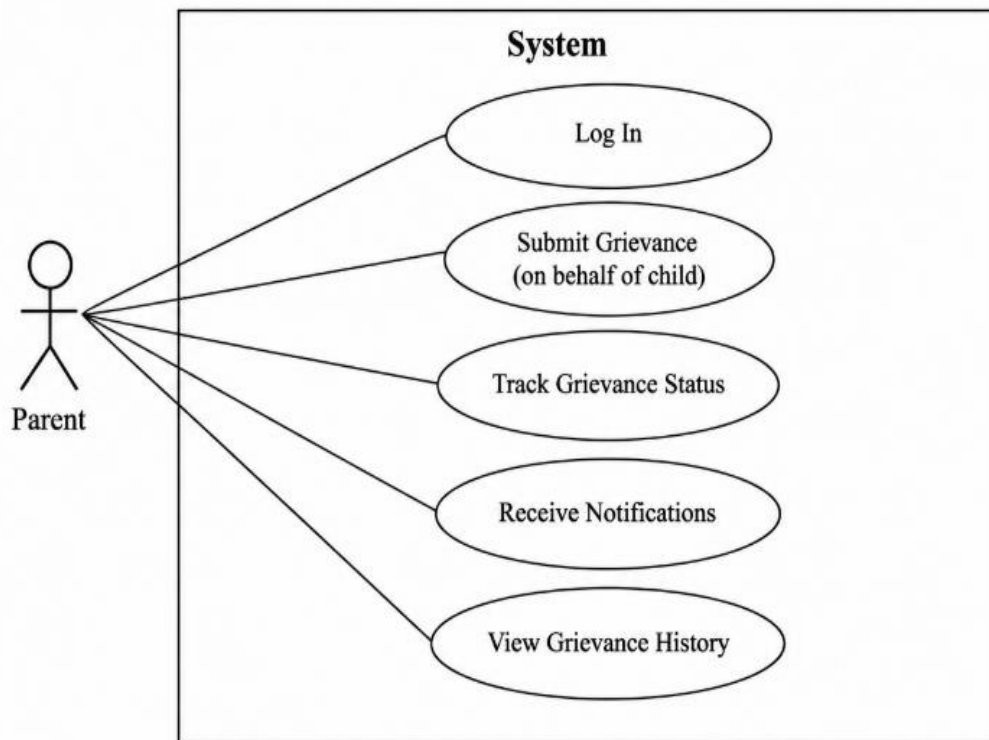
1. Missing information: Submission is rejected until required fields are completed.
2. Invalid attachment: System displays an attachment error.
3. Duplicate grievance: System warns the user when an existing grievance appears to match the submitted complaint.

UC-05: Track Student-Related Complaint

Primary Actor: Parent

Main Success Scenario:

1. Parent opens the tracking page.
2. Parent enters the complaint ID.
3. System retrieves the grievance.
4. System displays current status.
5. System displays previous actions and updates.



Teacher Use Case Description

UC-06: Submit School Grievance

Primary Actor: Teacher

Secondary Actor: Database Server

Trigger: Teacher selects "Submit Grievance".

Preconditions:

- Teacher is authenticated.

Postconditions:

- Grievance is stored and assigned a complaint ID.

Main Success Scenario:

1. Teacher logs into the portal.
2. Teacher opens the grievance form.
3. Teacher selects a grievance category.
4. Teacher enters the problem description.

5. Teacher attaches supporting documents when required.
6. Teacher submits the grievance.
7. System validates the information.
8. System generates the complaint ID.
9. System stores the grievance.
10. System notifies the responsible administrator.

Exceptions:

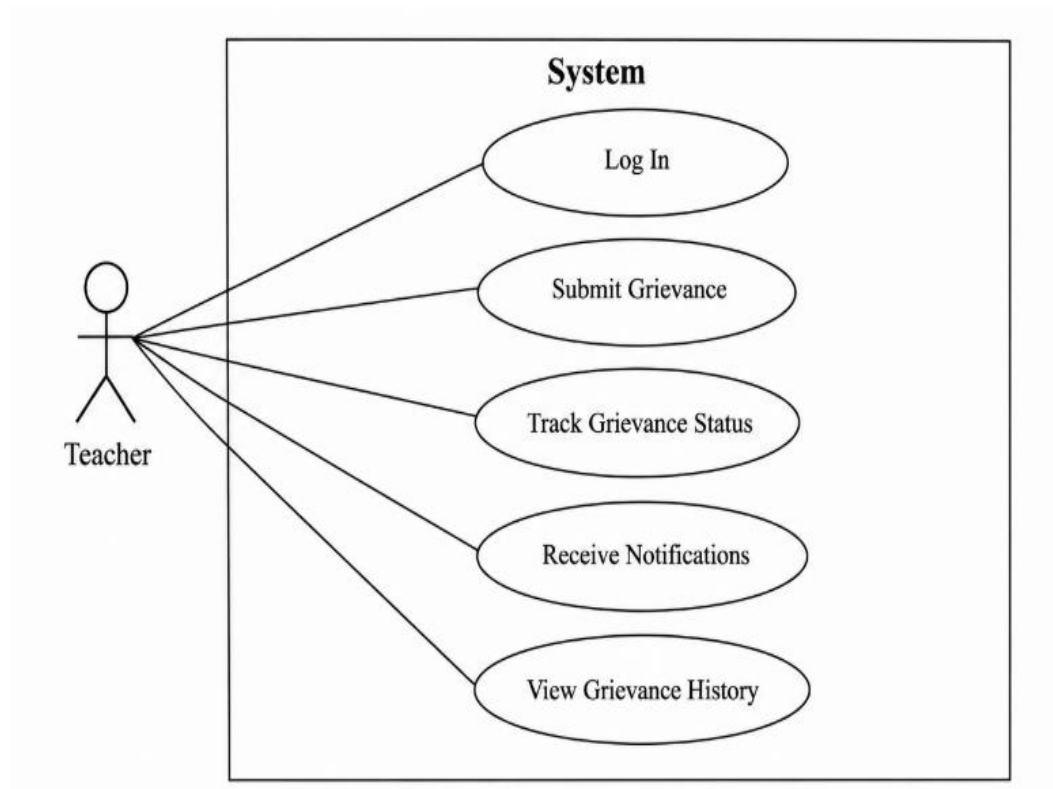
1. Missing mandatory information.
2. Invalid attachment.
3. Server/database connection failure.

UC-07: View Grievance Updates

Primary Actor: Teacher

Main Success Scenario:

1. Teacher opens the grievance dashboard.
2. System displays submitted grievances.
3. Teacher selects a grievance.
4. System displays its current status.
5. System displays administrator remarks and previous updates.



School Administrator Use Case Description

UC-08: Review Grievance

Primary Actor: School Administrator

Secondary Actor: Database Server

Trigger: Administrator opens the grievance management dashboard.

Preconditions:

- Administrator is authenticated.
- Grievances are available for review.

Postconditions:

- Grievance is reviewed and assigned an appropriate action.

Main Success Scenario:

1. Administrator logs into the portal.
2. System displays pending grievances.
3. Administrator selects a grievance.

4. System displays grievance details and attachments.
5. Administrator reviews the complaint.
6. Administrator identifies the responsible department/person.
7. Administrator assigns the grievance.
8. System records the assignment.

UC-09: Update Grievance Status

Primary Actor: School Administrator

Main Success Scenario:

1. Administrator selects a grievance.
2. Administrator reviews the current status.
3. Administrator selects a new status.
4. Administrator enters remarks if required.
5. System validates the update.
6. System stores the new status.
7. System records the status history.
8. System sends a notification to the concerned user.

Possible Status Values:

- Submitted
- Under Review
- Assigned
- In Progress
- Resolved
- Closed
- Reopened

Government Official Use Case Description

UC-10: Monitor Grievance Statistics

Primary Actor: Government Official

Secondary Actor: Analytics Module

Trigger: Government Official opens the monitoring dashboard.

Preconditions:

- Official is authenticated.
- Grievance data is available.

Postconditions:

- Statistics and analytical information are displayed.

Main Success Scenario:

1. Official logs into the portal.
2. System verifies authorization.
3. System retrieves grievance statistics.
4. System calculates category-wise and status-wise totals.
5. System displays dashboard charts.
6. Official reviews school-level grievance information.

UC-11: Generate Grievance Report

Primary Actor: Government Official

Secondary Actor: Reporting Module

Main Success Scenario:

1. Official selects the required reporting period.
2. Official selects school, category, or status filters.
3. System retrieves matching grievance records.
4. System calculates statistics.
5. System generates the report.
6. Official views the report.
7. Official downloads the report in PDF format.

Exception:

If insufficient data is available, the system displays a suitable message.

UC-12: Manage Users

Primary Actor: School Administrator

Preconditions:

- Administrator is authenticated.

Main Success Scenario:

1. Administrator opens the user management section.
2. Administrator selects Create, Update, or Deactivate.
3. Administrator enters user details.
4. System validates the information.
5. System updates the user database.
6. System records the administrative action.

Exceptions:

1. Duplicate account: System rejects duplicate registration.
2. Invalid information: System highlights the incorrect field.

UC-13: Manage Complaint Categories

Primary Actor: School Administrator

Main Success Scenario:

1. Administrator opens complaint category management.
2. Administrator creates or modifies a category.
3. System validates the category.
4. System stores the category.
5. Updated category becomes available in grievance forms.

Examples:

- Academic
- Infrastructure
- Sanitation
- Safety
- Student Welfare
- Transportation
- Administration
- Other

UC-14: Manage Notifications

Primary Actor: School Administrator

Secondary Actor: Notification Service

Main Success Scenario:

1. System detects a grievance status change.
2. System identifies the concerned user.
3. Notification message is generated.
4. Notification is delivered through the configured channel.
5. Notification history is stored.

UC-15: Review Complaint History

Primary Actor: School Administrator

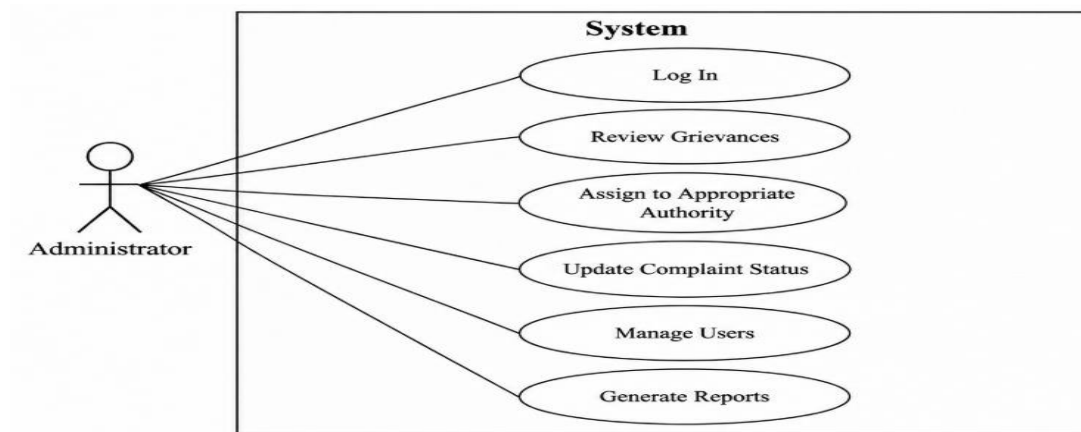
Secondary Actor: Database Server

Main Success Scenario:

1. Administrator selects a complaint.
2. System retrieves all recorded actions.
3. System displays submission details.
4. System displays assignment history.
5. System displays status changes.
6. System displays resolution remarks.
7. Administrator can review the complete grievance lifecycle.

Exceptions:

1. No history available: System displays the available grievance information.
2. Database unavailable: System displays a temporary service error



2.3 User Classes and Characteristics — Summary

Student

- Submit grievances
- Track complaint status
- Receive notifications
- View complaint history

Parent

- Register grievances related to students
- Track submitted complaints
- Receive status updates

Teacher

- Submit school-related grievances
- Track complaints
- View resolution updates

School Administrator

- Review grievances
- Assign complaints
- Update complaint status
- Manage users
- Manage complaint categories
- Generate administrative reports

Government Official

- Monitor grievance statistics

- View school-level information
- Generate analytical reports
- Monitor grievance resolution performance

2.4 Operating Environment

Hardware

- Desktop computers
- Laptop computers
- Mobile devices
- Standard server infrastructure
- Optional printer for report printing

Software

- Windows 10/11
- Linux
- Android
- Chrome
- Firefox
- Microsoft Edge
- MySQL Database
- Apache/XAMPP Server

2.5 Design and Implementation Constraints

- The system must follow applicable government regulations.
- Secure authentication must be implemented.
- Internet connectivity is required.
- User and grievance data must be protected.
- Role-based access control must be enforced.
- Database availability is required for normal operation.

2.6 User Documentation

- User Manual
- Administrator Guide

- Online Help System
- FAQ Section

2.7 Assumptions and Dependencies

- Users have internet access.
- Government school information is available.
- Authorized administrators manage grievances.
- The database server remains operational.
- Notification services are available when enabled.

3. External Interface Requirements

3.1 User Interfaces

Login Page

- Username
- Password
- Forgot Password
- Login button

Dashboard

- Complaint statistics
- Recent complaints
- Complaint status
- Notifications
- Quick grievance submission

Complaint Form

- Complaint category
- Complaint description
- School information
- Supporting document upload
- Submit button

Tracking Interface

- Complaint ID search
- Complaint status
- Status history
- Resolution remarks

3.2 Hardware Interfaces

- Desktop and laptop computers
- Mobile devices
- Server systems
- Printers for optional report output

The system does not require specialized hardware for normal operation.

3.3 Software Interfaces

Component Interface / Technology

Operating System Windows 10/11 or Linux

Web Server Apache/XAMPP

Frontend HTML, CSS, JavaScript

Database MySQL

Web Browser Chrome, Firefox, Edge

Email Service SMTP-based notification service

3.4 Communication Interfaces

- HTTP/HTTPS is used for communication between users and the web application.
- HTTPS should be used for secure transmission of sensitive information.

- Email notifications may be used for grievance status updates.
- SMS notification support may be integrated in future versions.
- Database communication is handled through the application server.
- Uploaded documents are transferred through secure application requests.

4. System Features

4.1 User Authentication and Authorization

4.1.1 Description and Priority

This feature allows users to securely register, authenticate, and access the Government Smart School Grievance Portal according to their assigned role.

Priority: High

4.1.2 Stimulus/Response Sequences

Stimulus: User enters username and password.

Response: System validates credentials and grants access when authentication succeeds.

Stimulus: User enters incorrect credentials.

Response: System displays an authentication error and prevents unauthorized access.

4.1.3 Functional Requirements

- REQ-001: The system shall allow user registration.
- REQ-002: The system shall allow secure login.
- REQ-003: The system shall support password recovery.
- REQ-004: The system shall provide role-based access control.
- REQ-005: The system shall prevent unauthorized access to restricted modules.
- REQ-006: The system shall terminate inactive sessions according to configured security rules.

4.2 Grievance Management

4.2.1 Description and Priority

This feature allows students, parents, and teachers to submit grievances electronically and allows authorized administrators to review and process them.

Priority: High

4.2.2 Stimulus/Response Sequences

Stimulus: User submits a grievance.

Response: System validates the grievance, generates a unique complaint ID, stores the grievance, and assigns it to the appropriate administrator.

Stimulus: User uploads supporting documentation.

Response: System validates and stores the document with the associated grievance.

4.2.3 Functional Requirements

- REQ-007: Users shall submit grievances online.
- REQ-008: System shall generate a unique complaint ID.
- REQ-009: System shall categorize grievances.
- REQ-010: System shall allow supporting document uploads.
- REQ-011: System shall maintain grievance history.
- REQ-012: Authorized administrators shall be able to assign grievances.
- REQ-013: System shall record every grievance status change.

4.3 Complaint Tracking

4.3.1 Description and Priority

The Complaint Tracking module allows users to monitor the current status and complete history of submitted grievances.

Priority: High

4.3.2 Stimulus/Response Sequences

Stimulus: User enters a valid complaint ID.

Response: System retrieves the corresponding grievance record and displays its current status and history.

4.3.3 Functional Requirements

- REQ-014: Users shall track grievances using complaint IDs.
- REQ-015: System shall display the current grievance status.
- REQ-016: System shall display grievance history.
- REQ-017: System shall display administrator remarks when available.
- REQ-018: System shall notify users when grievance status changes.

Possible Status Flow:

Submitted → Under Review → Assigned → In Progress → Resolved → Closed

A closed grievance may be reopened by an authorized administrator when further action is required.

4.4 Report Management

4.4.1 Description and Priority

The Report Management module generates statistical and analytical reports for school administrators and authorized government officials.

Priority: Medium

4.4.2 Stimulus/Response Sequences

Stimulus: Authorized official selects report criteria.

Response: System retrieves relevant grievance data, calculates statistics, and generates the requested report.

4.4.3 Functional Requirements

- REQ-019: System shall generate grievance reports.
- REQ-020: System shall provide dashboard analytics.
- REQ-021: System shall support filtering by school, category, date, and status.
- REQ-022: System shall provide grievance statistics.
- REQ-023: Reports shall be downloadable in PDF format.
- REQ-024: Only authorized users shall access administrative reports.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Login response time shall be less than 3 seconds under normal operating conditions.
- Complaint search results should be displayed within 2 seconds.
- The system should support at least 1000 concurrent users.
- Standard reports should be generated within 5 seconds for normal data volumes.

5.2 Safety Requirements

- The system shall prevent accidental deletion of grievance records.
- Backup and recovery mechanisms shall be provided.
- User input shall be validated before processing.
- Critical administrative operations shall require authorization.

5.3 Security Requirements

- Passwords shall be securely stored.
- HTTPS communication shall be supported.
- Role-based access control shall be implemented.
- SQL Injection attacks shall be prevented.
- Cross-Site Scripting attacks shall be prevented.
- Secure session management shall be implemented.
- Unauthorized access shall be restricted.

5.4 Software Quality Attributes

Reliability

The system should provide reliable grievance registration, tracking, and reporting services.

Availability

System availability should be approximately 99% during normal service periods.

Usability

The portal should provide a simple and user-friendly interface for students, parents, teachers, administrators, and government officials.

Maintainability

The system should use a modular architecture so that future changes can be implemented easily.

Scalability

The system should support increasing numbers of users, schools, grievances, and reports.

Portability

The application should be accessible through desktops, tablets, and mobile devices using standard web browsers.

5.5 Business Rules

- Every grievance must have a unique complaint ID.
- Only authorized users can update grievance status.
- Only authorized officials can close complaints.
- Every status update must be recorded.
- Grievance records must be retained for audit purposes.
- Users can access only grievances permitted by their role.

6. Other Requirements

6.1 Database Requirements

Tables:

- Users
- Complaints
- Complaint Categories
- Complaint Status
- Notifications
- Reports
- Schools

Primary keys and foreign keys shall be used to maintain data integrity.

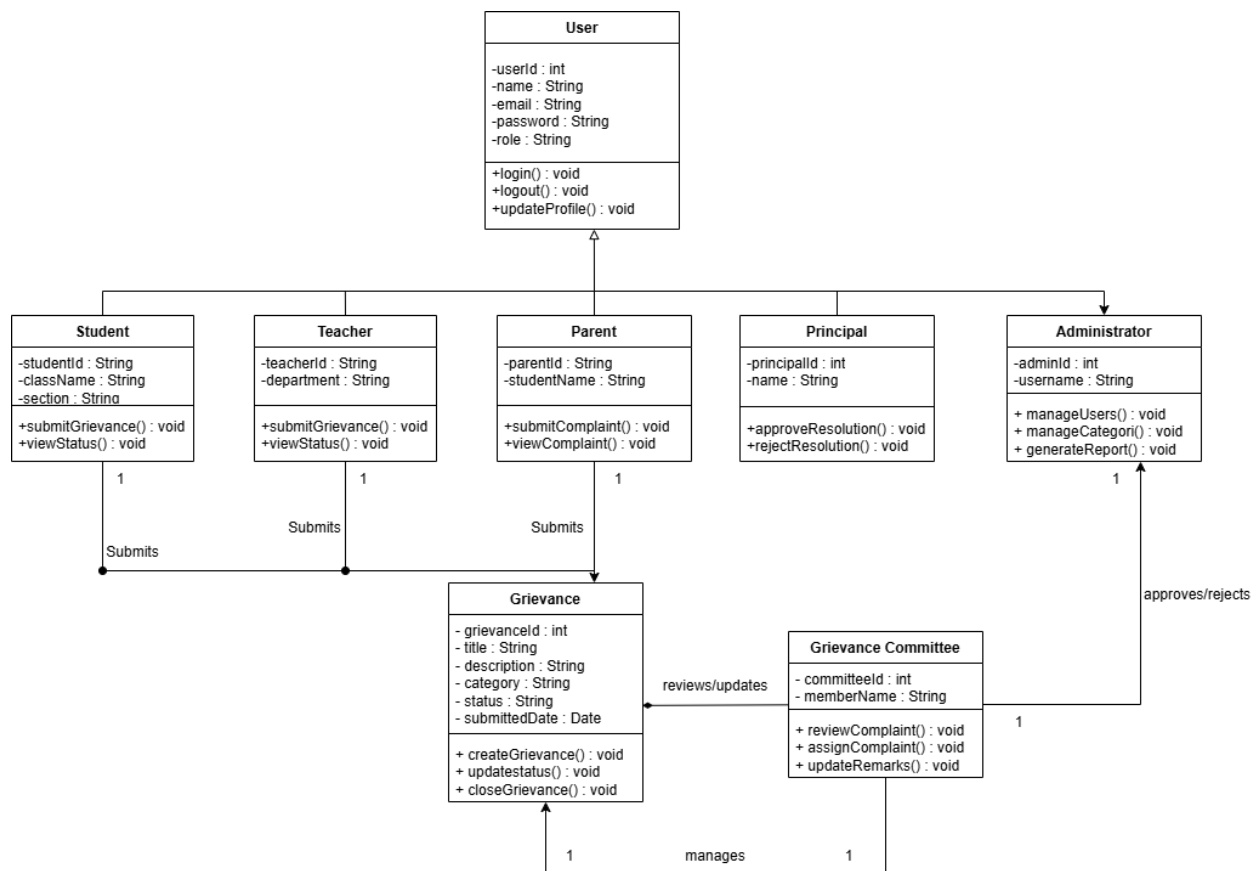
6.2 Legal Requirements

- Compliance with applicable government educational regulations.
- Compliance with applicable data privacy policies.

Appendix B: Analysis Models

B.1 UML Class Diagram

Figure B.1: UML Class Diagram of Government Smart School Grievance Portal



B.1 UML Class Diagram

The UML Class Diagram represents the structural model of the Government Smart School Grievance Portal. It identifies the major classes, their attributes, operations, and relationships.

The main entities include User, Student, Parent, Teacher, SchoolAdministrator, GovernmentOfficial, Complaint, ComplaintCategory, ComplaintStatus, Notification, School, and Report.

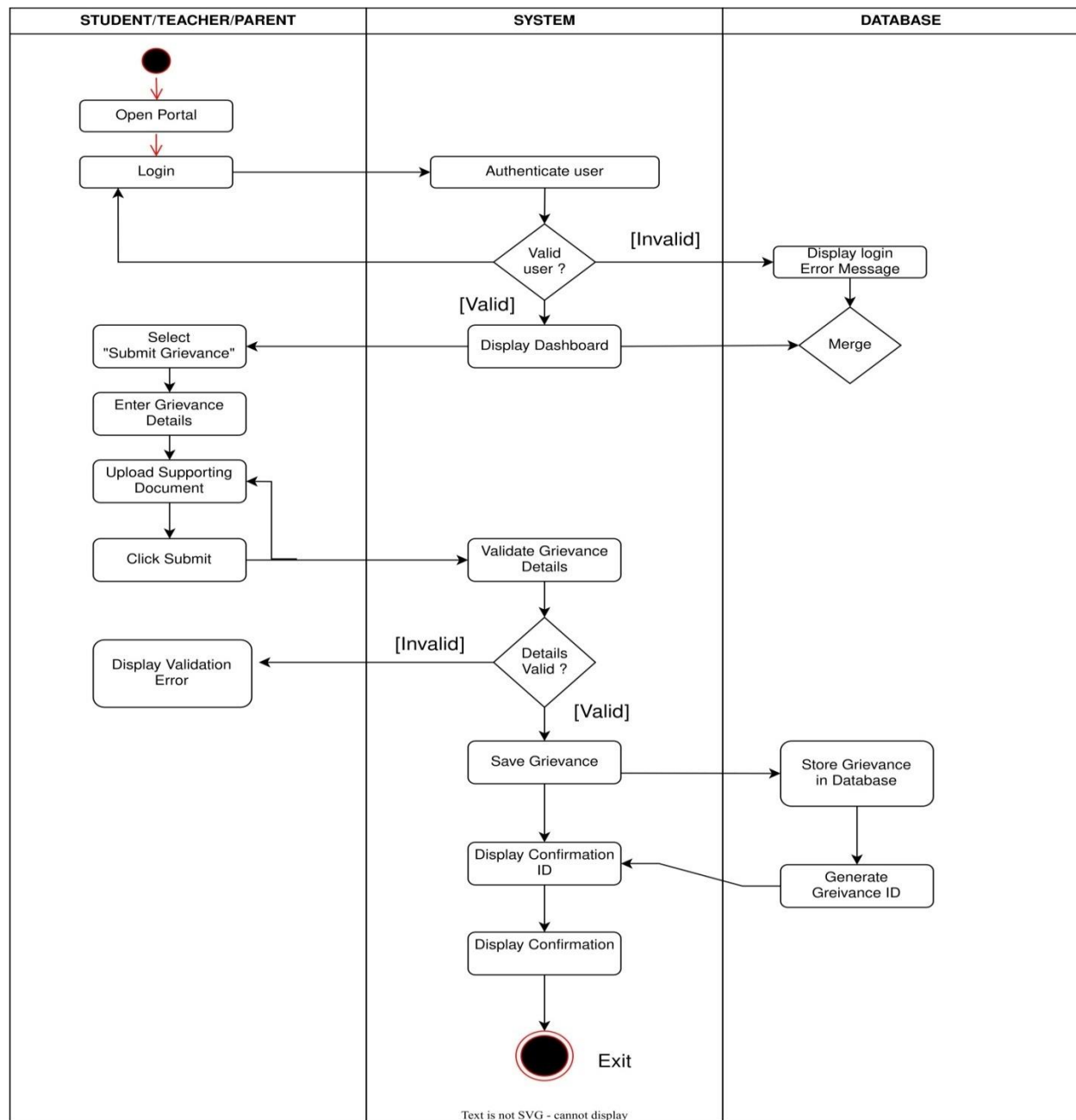
The relationships between these entities represent user registration, grievance submission, assignment, status tracking, notification, and reporting operations.

B.1.1 Class Dictionary and Data Definitions

B.1.1 Class Dictionary and Data Definitions			
Class	Responsibility	Attributes	Operations / Methods
 User	Provides common authentication and profile information.	• userId • name • email • password • role	• login() • logout() • updateProfile()
 Student	Represents students who submit and track grievances.	• studentId • className • schoolId	• submitGrievance() • trackGrievance()
 Parent	Represents parents who register student-related grievances.	• parentId • studentId • contactNumber	• submitGrievance() • trackGrievance()
 Teacher	Represents teachers who submit school-related grievances.	• teacherId • schoolId • department	• submitGrievance() • viewUpdates()
 SchoolAdministrator	Manages grievances and user records within a school.	—	• reviewComplaint() • assignComplaint() • updateStatus() • manageUsers()
 GovernmentOfficial	Monitors grievance statistics and generates reports.	—	• viewAnalytics() • generateReport()
 Complaint	Represents a grievance submitted by a user.	• complaintId • title • description • category • submittedDate • status	• createComplaint() • updateStatus() • viewHistory()
 ComplaintCategory	Defines the category/type of a complaint.	• categoryId • categoryName	—
 Notification	Stores notifications sent to users regarding grievances.	• notificationId • message • date • status	—
 Report	Stores generated grievance reports and related information.	• reportId • reportType • generatedDate • data	—

B.2 Activity Diagram

Figure B.2: UML Activity Diagram – Submit Grievance Module



B.2.1 Submit Grievance Activity Diagram

The activity diagram represents the workflow followed when a student, parent, or teacher submits a grievance.

The process begins with user authentication and continues through grievance form entry, validation, complaint registration, complaint ID generation, database storage, and submission confirmation.

Preconditions:

- User is authenticated.
- Grievance submission service is available.

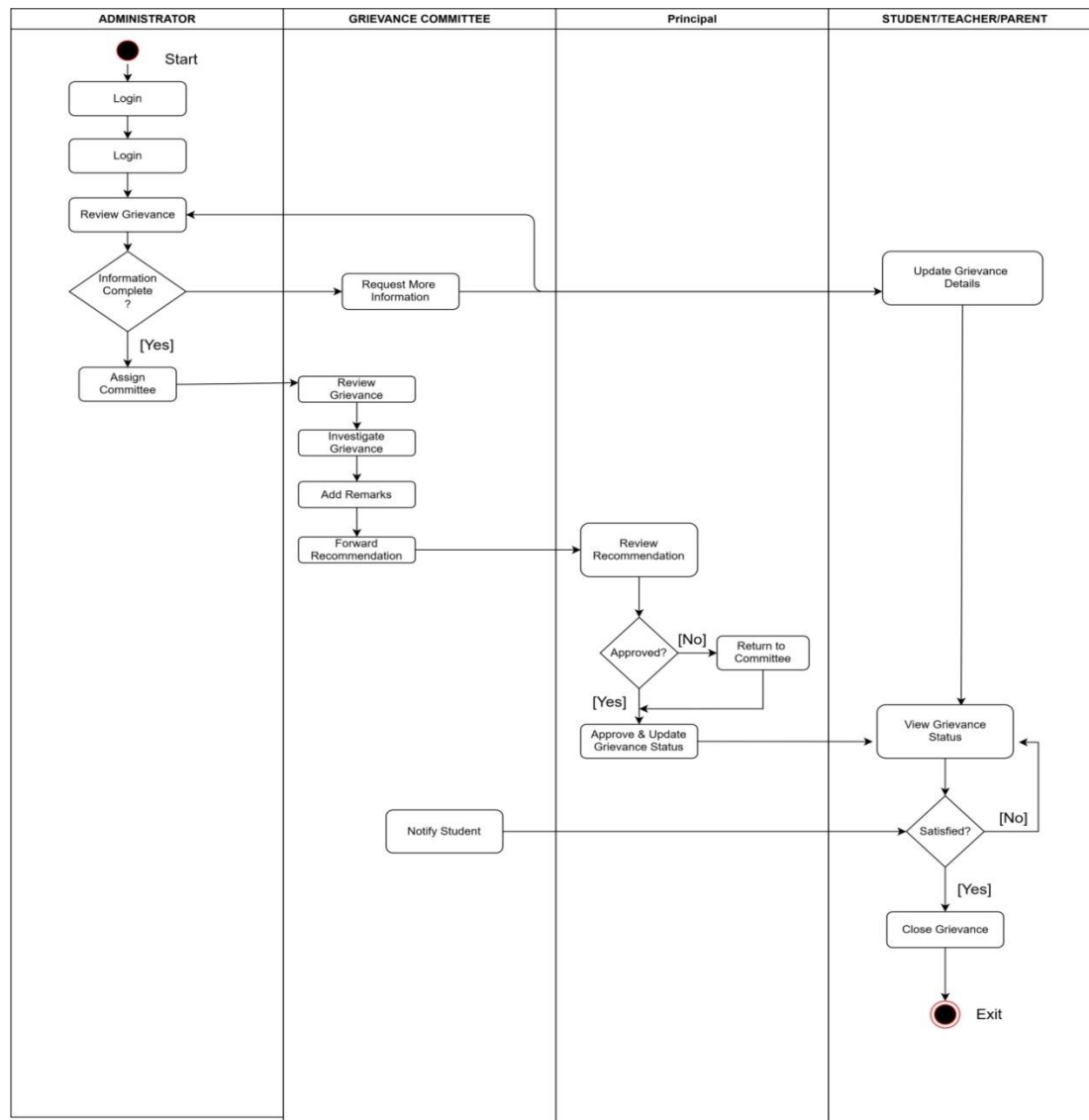
Postconditions:

- Grievance is stored successfully.
- Unique complaint ID is generated.
- User receives confirmation.

Execution Sequence

1. User logs into the portal.
2. System verifies authentication.
3. User opens the grievance form.
4. User selects a complaint category.
5. User enters grievance details.
6. User uploads supporting documents if necessary.
7. User submits the grievance.
8. System validates the submitted information.
9. If validation fails, the user is asked to correct the information.
10. If validation succeeds, the system generates a complaint ID.
11. System stores the grievance.
12. System assigns the initial status as "Submitted".
13. System sends confirmation to the user.
14. The workflow ends successfully.

Figure B.3: UML Activity Diagram – Review and Resolve Grievance Module



B.2.2 Review and Resolve Grievance Activity Diagram

This activity diagram represents the workflow performed by a School Administrator when reviewing and resolving a submitted grievance.

Preconditions:

- Administrator is authenticated.
- A submitted grievance exists.

Postconditions:

- Grievance is resolved or returned for additional action.
- All actions are recorded in the grievance history.

The workflow includes reviewing the complaint, assigning the responsible person, updating the status, recording resolution remarks, and closing or reopening the grievance.

Execution Sequence

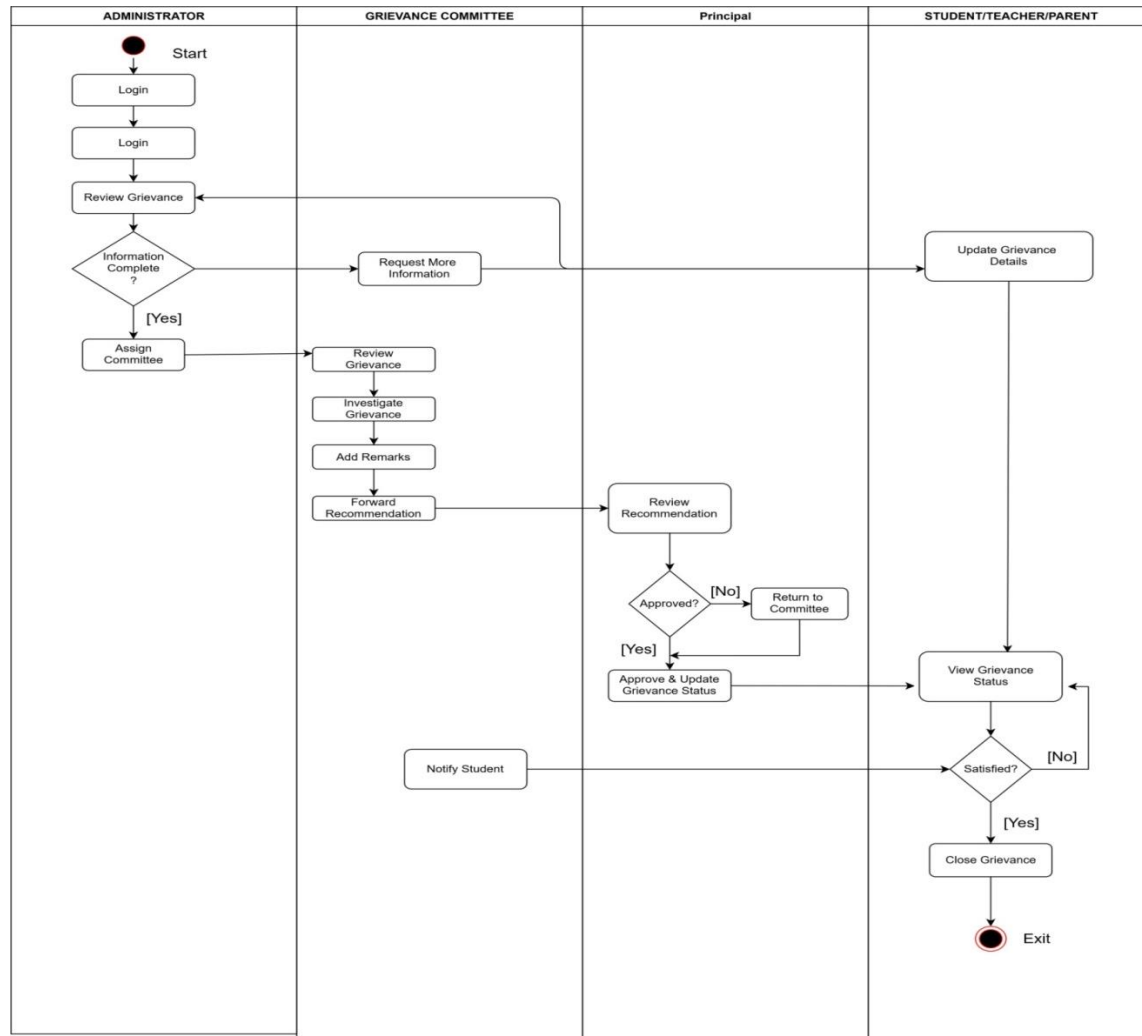
1. Administrator logs into the system.
2. System displays pending grievances.
3. Administrator selects a grievance.
4. System displays complete grievance information.
5. Administrator reviews the complaint and supporting documents.
6. Administrator assigns the complaint to the responsible department or person.
7. Status changes to "Assigned".
8. Responsible person begins corrective action.
9. Status changes to "In Progress".
10. Resolution details are entered.
11. Administrator verifies the resolution.
12. If the resolution is acceptable, status changes to "Resolved".
13. Authorized administrator closes the grievance.
14. System records the final action.
15. User receives the resolution notification.

Exception:

If the complaint is not satisfactorily resolved, the grievance may be reopened for further action.

B.3 Grievance State / Workflow Diagram

Figure B.4: Grievance State / Workflow Diagram



The state diagram represents the complete lifecycle of a grievance from initial submission to final closure.

Primary States:

- Submitted
- Under Review
- Assigned

- In Progress
- Resolved
- Closed
- Reopened

The state transitions occur based on administrator actions and grievance processing events.

Behavioral Transition Logic

1. Entry State

A new grievance enters the "Submitted" state after successful registration.

2. Review Transition

The administrator reviews the grievance and moves it to "Under Review".

3. Assignment Transition

After review, the grievance is assigned to the appropriate person or department.

4. Processing Transition

The grievance enters "In Progress" while corrective action is being performed.

5. Resolution Transition

After corrective action, the complaint is marked "Resolved".

6. Closure Transition

An authorized administrator verifies the resolution and changes the state to "Closed".

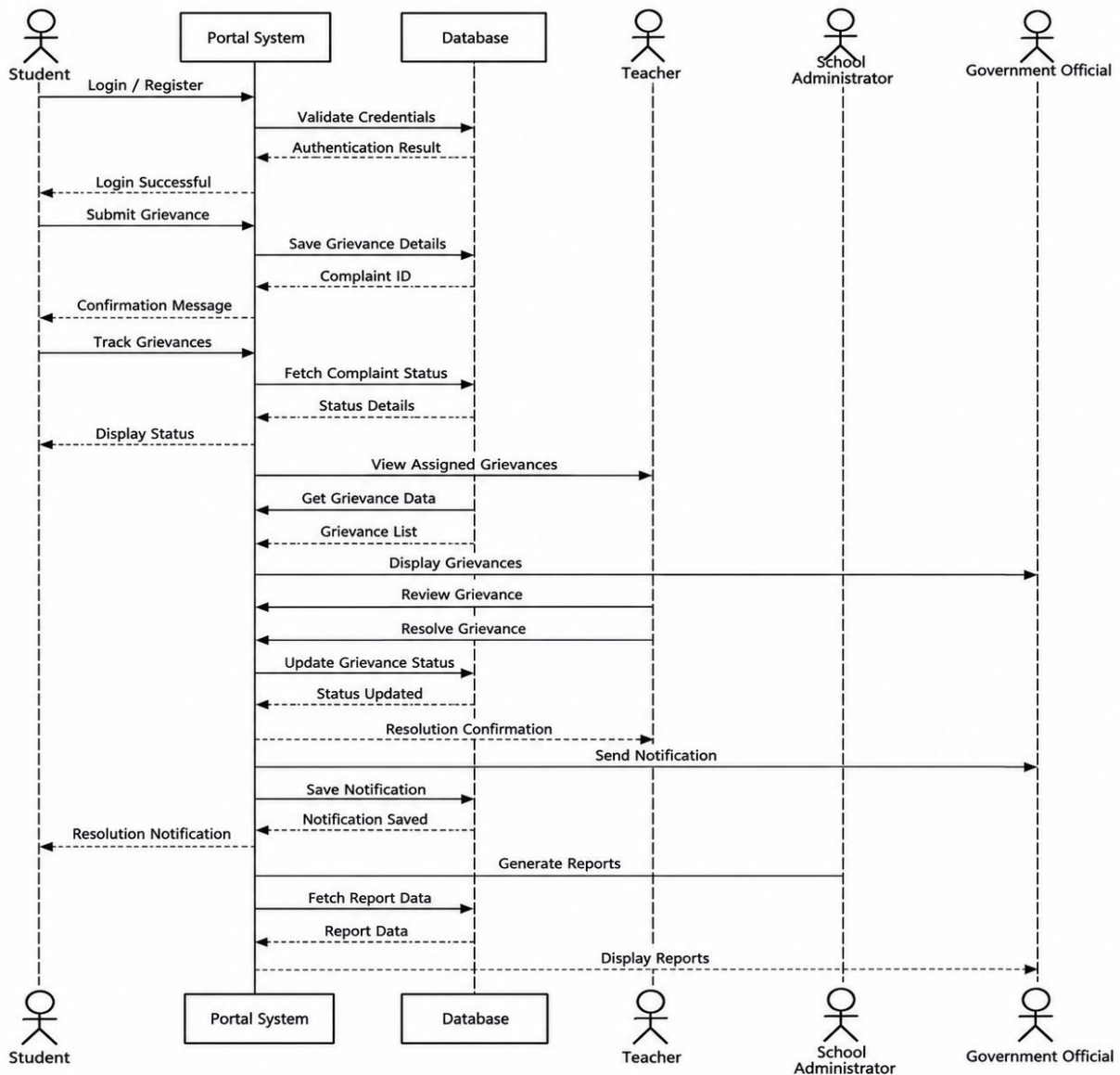
7. Reopening Transition

If additional action is required, an authorized administrator can reopen the complaint.

The workflow ensures that every important state transition is recorded for transparency and audit purposes.

B.4 Sequence Diagram

Figure B.5: Sequence Diagram – Grievance Submission



B.4.1 Grievance Submission Sequence Diagram

Purpose

This sequence diagram illustrates the interaction between the user, web interface, grievance controller, database, and notification service during grievance submission.

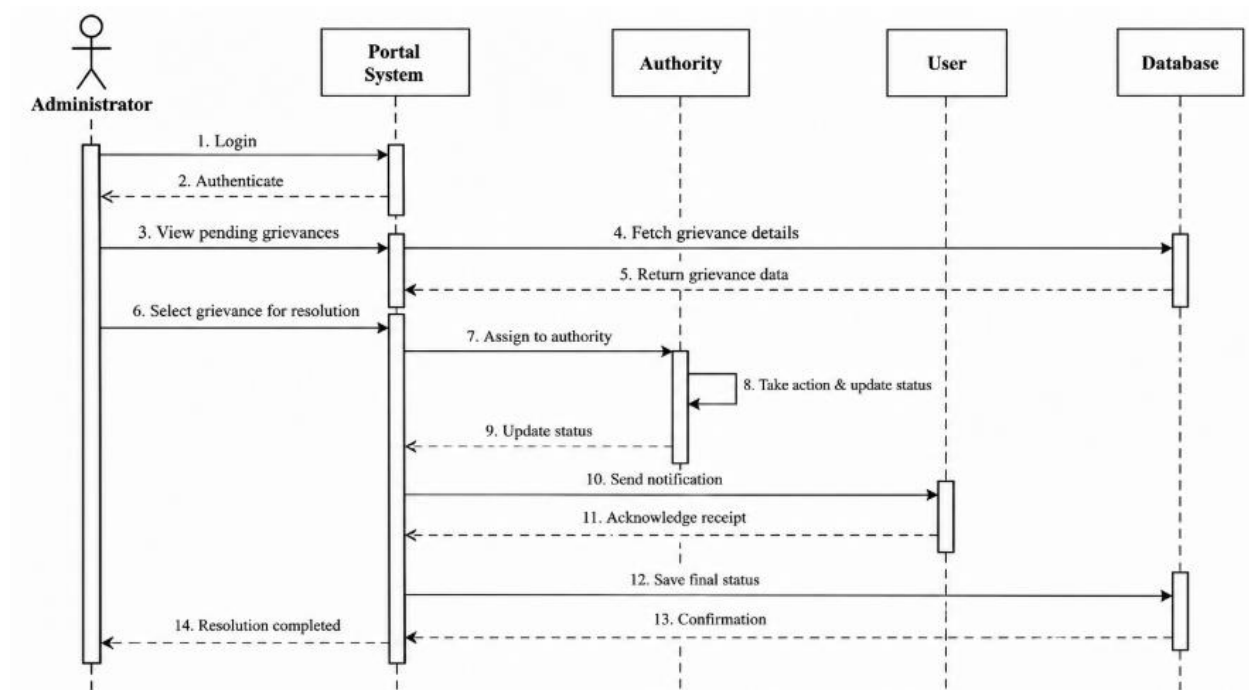
Participants

- User
- Web Interface
- Grievance Controller
- Database
- Notification Service

Execution Flow

1. User opens grievance form.
2. Web interface displays the form.
3. User enters grievance information.
4. Web interface sends the information to the grievance controller.
5. Controller validates the submitted data.
6. Controller generates a complaint ID.
7. Complaint information is stored in the database.
8. Database confirms successful storage.
9. Notification service sends submission confirmation.
10. System displays the complaint ID to the user.

Figure B.6: Sequence Diagram – Complaint Resolution



B.4.2 Complaint Resolution Sequence Diagram

Purpose

This sequence diagram represents the interaction between the administrator, grievance management module, database, and notification service during complaint resolution.

Participants

- School Administrator
- Grievance Management Module
- Database
- Notification Service
- User

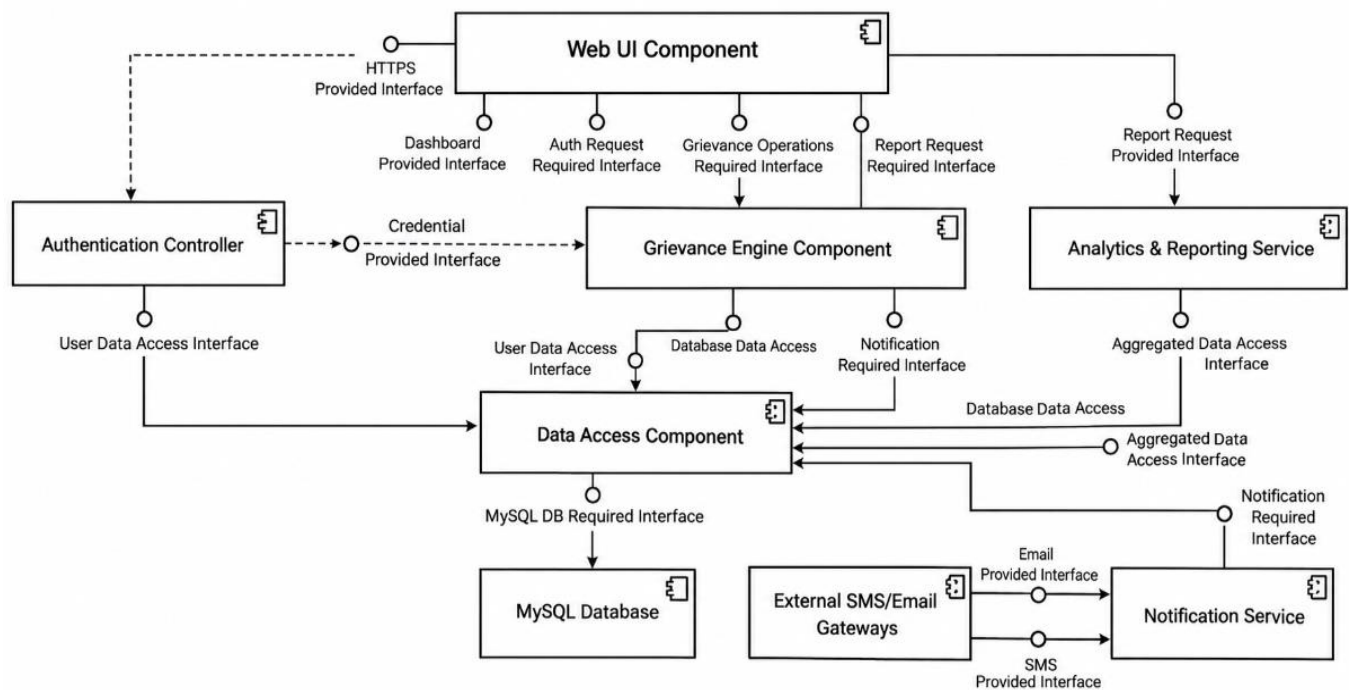
Execution Flow

1. Administrator selects a pending grievance.
2. System retrieves grievance details.
3. Administrator reviews the complaint.
4. Administrator assigns the grievance.
5. Database stores the assignment.

6. Administrator updates the grievance status.
7. System stores the status change.
8. Resolution information is entered.
9. System updates the grievance record.
10. Notification service informs the user.
11. User views the updated grievance status.

B.6 UML Component Diagram

Figure B.7: UML Component Diagram of Government Smart School Grievance Portal



The component diagram illustrates the major software modules of the Government Smart School Grievance Portal and the relationships among them.

Major Components:

- Authentication Component
- User Management Component
- Grievance Management Component
- Complaint Tracking Component
- Notification Component

- Report and Analytics Component
- Database Component

These components communicate through defined interfaces to provide the complete grievance management functionality.

B.5.1 Components of the System

Authentication Component

Handles user registration, login, password recovery, authentication, and role-based access control.

User Management Component

Maintains user profiles, roles, school information, and account status.

Grievance Management Component

Handles grievance registration, categorization, document upload, assignment, and processing.

Complaint Tracking Component

Allows users to search for complaints and view their current status and history.

Notification Component

Generates and sends notifications when grievances are submitted, assigned, updated, resolved, or closed.

Report and Analytics Component

Generates statistical reports and dashboard information for authorized administrators and government officials.

Database Component

Provides persistent storage for users, complaints, categories, statuses, notifications, schools, and reports.

B.5.2 Actors and Component Interaction

Students, Parents, Teachers, School Administrators, and Government Officials interact with the appropriate components according to their assigned roles.

B.6 UML Deployment Diagram

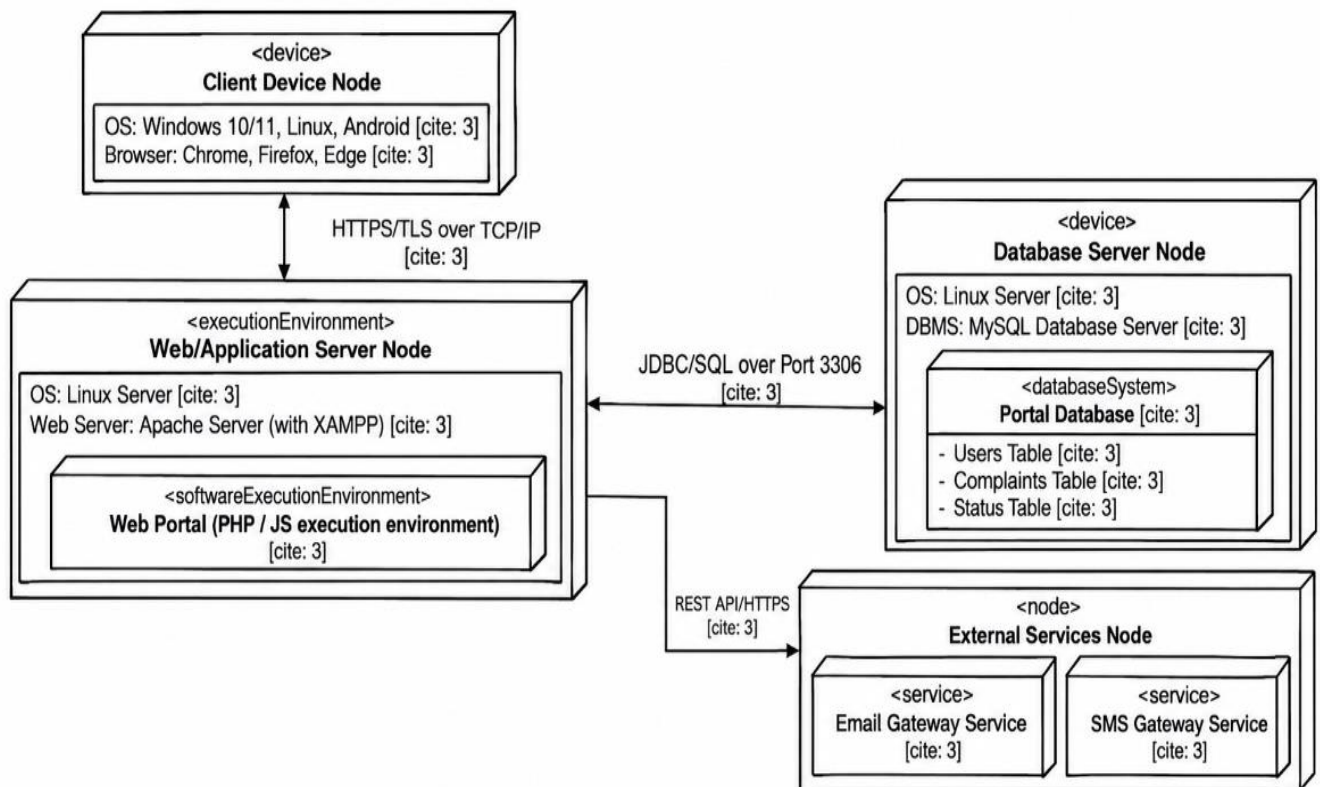


Figure B.9: UML Deployment Diagram of Government Smart School Grievance Portal

The UML Deployment Diagram represents the physical deployment of the Government Smart School Grievance Portal across client devices, web servers, application services, databases, and supporting services.

The deployment architecture follows a multi-tier structure consisting of:

- Client Tier
- Web Tier
- Application Tier

- Data Tier
- Notification Services

This architecture separates presentation, application logic, data management, and supporting services.

B.6.1 Deployment Nodes

Client Device

Represents computers, laptops, tablets, and smartphones used by students, parents, teachers, administrators, and government officials.

Web Server

Hosts the web interface and handles HTTP/HTTPS requests from client devices.

Application Server

Executes the core application logic for authentication, grievance processing, tracking, notifications, and reporting.

Database Server

Hosts the MySQL database containing users, complaints, categories, statuses, notifications, schools, and reports.

File Storage

Stores supporting documents uploaded with grievances.

Notification Server

Provides email or SMS notification services.

Backup Storage

Maintains backup copies of important grievance and system data.

B.6.2 Deployed Artifacts

- Government Smart School Grievance Portal Web UI
- Application Services
- MySQL Database
- Uploaded Supporting Documents
- Notification Services
- Backup Data

B.7.IMPLEMENTATION

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width,
initial-scale=1.0">

  <title>
    Government Smart School
    Grievance Portal
  </title>

  <link rel="stylesheet"
    href="styles.css">
</head>

<body>

<header class="navbar">

  <div class="logo">
    Government Smart School
    Grievance Portal
  </div>

  <nav>

    <a href="#home">Home</a>

    <a href="#about">About</a>

    <a
      href="#services">Services</a>

    <a href="#login">Login</a>

  </nav>
</header>

<section id="home"
  class="hero">

  <div class="hero-content">

    <h1>
      Government Smart School
      Grievance Portal
    </h1>

    <p>
      A centralized digital platform
      for registering, tracking and
```

resolving school-related
grievances.

</p>

<button
onclick="showGrievanceForm()">
Submit a Grievance
</button>

</div>

</section>

<section id="about"
class="section">

<h2>About the Portal</h2>

<p>

</p>

</section>

<section id="services"
class="section">

<h2>Services</h2>

<div class="service-container">

<div class="service-card">

<h3>
Submit Grievance
</h3>

<p>

Register a new school-
related
grievance online.

</p>

</div>

<div class="service-card">

<h3>
Track Grievance
</h3>

<p>
Track the current status
of an existing complaint.
</p>

</div>

<div class="service-card">

<h3>
View Updates
</h3>

<p>
View status changes and
administrator remarks.
</p>

</div>

<div class="service-card">

<h3>
Reports

```

    </h3>

    <p>
        Authorized officials can
        generate grievance reports.
    </p>

</div>

</div>

</section>

<section id="login"
    class="section">

    <h2>User Login</h2>

    <form id="loginForm">

        <label>Email</label>

        <input type="email"
            id="loginEmail"
            required>

        <label>Password</label>

        <input type="password"
            id="loginPassword"
            required>

        <button type="submit">
            Login
        </button>

    </form>

    <p id="loginMessage"></p>

</section>

<section id="grievanceForm"
    class="section hidden">

    <h2>Submit Grievance</h2>

    <form id="complaintForm">

        <label>Name</label>

        <input type="text"
            id="name"
            required>

        <label>Email</label>

        <input type="email"
            id="email"
            required>

        <label>School</label>

        <input type="text"
            id="school"
            required>

        <label>Category</label>

        <select id="category"
            required>

            <option value="">
```

Select Category	
</option>	<label>
	Complaint Description
<option value="Academic">	</label>
Academic	
</option>	<textarea id="description"
	rows="6"
<option	required>
value="Infrastructure">	</textarea>
Infrastructure	
</option>	
	<button type="submit">
<option value="Sanitation">	Submit Grievance
Sanitation	</button>
</option>	
	</form>
<option value="Safety">	
Safety	<div
</option>	id="complaintMessage"></div>
<option value="Student	</section>
Welfare">	
Student Welfare	
</option>	<section class="section">
<option	Track Grievance
value="Transportation">	
Transportation	<form id="trackingForm">
</option>	
	<input type="text"
<option	id="complaintId"
value="Administration">	placeholder="Enter
Administration	Complaint ID"
</option>	required>
<option value="Other">	
Other	<button type="submit">
</option>	Track
	</button>
</select>	
	</form>

<div id="trackingResult"></div>	color: #333;
</section>	}
<footer>	.navbar {
<p>	background: #1d4ed8;
© 2026 Government Smart	color: white;
School	display: flex;
Grievance Portal	justify-content: space-between;
</p>	align-items: center;
</footer>	padding: 18px 50px;
<script src="app.js"></script>	}
</body>	
</html>	
Frontend Styling	.logo {
* {	font-size: 20px;
margin: 0;	font-weight: bold;
padding: 0;	}
box-sizing: border-box;	
}	.navbar nav a {
	color: white;
body {	text-decoration: none;
font-family: Arial, sans-serif;	margin-left: 25px;
background: #f5f7fa;	

```
}                                margin-bottom: 30px;

                                }

.hero {

    min-height: 500px;

    display: flex;

    justify-content: center;

    align-items: center;

    text-align: center;

}

.hero-content {

    max-width: 700px;

    padding: 30px;

}

.hero h1 {

    font-size: 42px;

    margin-bottom: 20px;

}

.hero p {

    font-size: 18px;

                                button {

                                    background: #2563eb;

                                    color: white;

                                    border: none;

                                    padding: 12px 25px;

                                    border-radius: 6px;

                                    cursor: pointer;

                                    font-size: 16px;

                                }

                                .section {

                                    padding: 60px 10%;

                                    text-align: center;

                                }

                                .section h2 {

                                    margin-bottom: 25px;

                                }
```

```
.service-container {
    border-radius: 10px;
    display: grid;
    grid-template-columns:
        repeat(
            auto-fit,
            minmax(220px, 1fr)
        );
    gap: 25px;
    margin-top: 30px;
}

.service-card {
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow:
        0 4px 12px
        rgba(0,0,0,0.1);
}

form {
    max-width: 600px;
    margin: auto;
    background: white;
    padding: 30px;

    input,
    select,
    textarea {
        width: 100%;
        padding: 12px;
        border: 1px solid #ccc;
        border-radius: 5px;
        font-size: 15px;
    }

    label {
        text-align: left;
        font-weight: bold;
    }
}
```

```
.hidden {  
  
    display: none;  
  
}  
footer {  
  
    background: #1e293b;  
  
    color: white;  
  
    text-align: center;  
  
    padding: 25px;  
  
    margin-top: 40px;  
  
}  
  
@media (max-width: 768px) {  
  
    .navbar {  
  
        flex-direction: column;  
  
        gap: 15px;  
  
        padding: 20px;  
  
    }  
  
    .navbar nav a {  
  
        margin: 8px;  
  
    }  
}
```

```
.hero h1 {  
  
    font-size: 30px;  
  
}  
}  
Frontend JavaScript  
const API_URL = "api/index.php";  
  
function showGrievanceForm() {  
  
    const form =  
        document.getElementById(  
            "grievanceForm"  
        );  
  
    form.classList.remove("hidden");  
  
    form.scrollIntoView({  
        behavior: "smooth"  
    });  
}  
  
document  
    .getElementById("loginForm")  
    .addEventListener(  
        "submit",  
        async function(event) {  
  
            event.preventDefault();  
  
            const email =  
                document.getElementById(  
                    "loginEmail"  
                ).value;
```

```
const password =
  document.getElementById(
    "loginPassword"
  ).value;

try {

  const response =
    await fetch(
      API_URL,
      {

        method: "POST",

        headers: {
          "Content-Type":
            "application/json"
        },

        body:
          JSON.stringify({

            action: "login",

            email: email,

            password:
              password

          })

      }
    );

  const result =
    await response.json();

  document.getElementById(
    "loginMessage"
  ).innerText =
    result.message;

} catch (error) {

  document.getElementById(
    "loginMessage"
  ).innerText =
    "Server connection
    error.";

}

document
  .getElementById("complaintForm")
  .addEventListener(
    "submit",
    async function(event) {

      event.preventDefault();

      const complaint = {

        action:
          "submit_complaint",

        name:

      }

      document.getElementById(
        "loginMessage"
      ).innerText =
        "Server connection
        error.";

    }
  );
```

```
        "name"
    ).value,
    email:
        document.getElementById(
            "email"
        ).value,
    school:
        document.getElementById(
            "school"
        ).value,
    category:
        document.getElementById(
            "category"
        ).value,
    description:
        document.getElementById(
            "description"
        ).value
    };

    try {
        const response =
            await fetch(
                API_URL,
                {
                    method: "POST",
                    headers: {
                        "Content-Type":
                            "application/json"
                    },
                    body:
                        JSON.stringify(
                            complaint
                        )
                }
            );

        const result =
            await response.json();

        document.getElementById(
            "complaintMessage"
        ).innerText =
            result.message;

    } catch (error) {
        document.getElementById(
            "complaintMessage"
        ).innerText =
            "Unable to submit
            grievance.";
    }

    }
    }
    );

    document
        .getElementById("trackingForm")
```

```
.addEventListener(
    "submit",
    async function(event) {
        event.preventDefault();

        const complaintId =
            document.getElementById(
                "complaintId"
            ).value;

        try {
            const response =
                await fetch(
                    `${API_URL}?action=track&id=${complaintId}`
                );

            const result =
                await response.json();

            const output =

            document.getElementById(
                "trackingResult"
            );

            if (result.success) {
                output.innerHTML = `
                    <h3>
                        Complaint Details
                    </h3>
                    <p>
                        Complaint ID:
                        ${result.data.complaint_id}
                    </p>
                    <p>
                        Category:
                        ${result.data.category}
                    </p>
                    <p>
                        Status:
                        ${result.data.status}
                    </p>
                    <p>
                        Description:
                        ${result.data.description}
                    </p>
                `;
            } else {
                output.innerText =
                    result.message;
            }
        } catch (error) {
            document.getElementById(
                "trackingResult"
            ).innerText =
```

```
        "Unable to retrieve  
grievance.";
```

```
    }
```

```
    }
```

```
);
```

Backend Implementation

api/config.php

```
<?php
```

```
$host = "localhost";
```

```
$dbname = "grievance_portal";
```

```
$username = "root";
```

```
$password = "";
```

```
try {
```

```
    $pdo = new PDO(
```

```
        "mysql:host=$host;  
dbname=$dbname;  
charset=utf8mb4",
```

```
        $username,
```

```
        $password,
```

```
        [
```

```
            PDO::ATTR_ERRMODE =>
```

```
            PDO::ERRMODE_EXCEPTION,
```

```
            PDO::ATTR_DEFAULT_FETCH_M  
ODE =>
```

```
                PDO::FETCH_ASSOC
```

```
        ]
```

```
    );
```

```
} catch (PDOException $e) {
```

```
    http_response_code(500);
```

```
    echo json_encode([
```

```
        "success" => false,
```

```
        "message" =>
```

```
            "Database connection failed."
```

```
    ]);
```

```
    exit;
```

```
}
```

```
?>
```

Backend API

api/index.php

```
<?php
```

```
header(
```

```
    "Content-Type: application/json"
```

```
);
```

```
require_once "config.php";
```

```
$method =
```

```
$_SERVER["REQUEST_METHOD"]  
];
```

```
if ($method === "POST") {
```

```
    $input =  
        json_decode(  
            file_get_contents(  
                "php://input"  
            ),  
            true  
        );
```

```
    /*  
    User Login  
    */
```

```
    if (  
        isset($input["action"]) &&  
        $input["action"] === "login"  
    ) {
```

```
        $email =  
            $input["email"] ?? "";
```

```
        $password =  
            $input["password"] ?? "";
```

```
        $stmt =  
            $pdo->prepare(  
                "SELECT *  
                FROM users  
                WHERE email = ?"  
            );
```

```
        $stmt->execute([  
            $email  
        ]);
```

```
        $user =  
            $stmt->fetch();
```

```
        if (  
            $user &&  
            password_verify(  
                $password,  
                $user["password"]  
            )  
        ) {
```

```
            echo json_encode([  
                "success" => true,  
                "message" =>  
                    "Login successful.",
```

```
                "user" => [  
                    "id" =>  
                        $user["id"],  
                    "name" =>  
                        $user["name"],  
                    "email" =>  
                        $user["email"],  
                    "role" =>  
                        $user["role"]
```

```
                ]
```

```
    });
} else {

    echo json_encode([

        "success" => false,

        "message" =>
            "Invalid email or
password."

    ]);

}

exit;

}

/*
    Submit Grievance
*/

if (
    isset($input["action"]) &&
    $input["action"] ===
    "submit_complaint"
) {

    $name =
        $input["name"] ?? "";

    $email =
        $input["email"] ?? "";

    $school =
        $input["school"] ?? "";
```

```
    $category =
        $input["category"] ?? "";

    $description =
        $input["description"] ?? "";

    if (
        empty($name) ||
        empty($email) ||
        empty($school) ||
        empty($category) ||
        empty($description)
    ) {

        echo json_encode([

            "success" => false,

            "message" =>
                "All required fields must
be completed."

        ]);

        exit;

    }

    $complaintId =
        "GRV-" .
        date("Ymd") .
        "-" .
        strtoupper(
            substr(
                uniqid(),
                -6
            )
        );
```

```

    );

$stmt =
    $pdo->prepare(
        "INSERT INTO complaints
        (
            complaint_id,
            name,
            email,
            school,
            category,
            description,
            status,
            created_at
        )
        VALUES
        (
            ?, ?, ?, ?, ?, ?,
            'Submitted',
            NOW()
        )"
    );

    /*
    Track Grievance
    */

$stmt->execute([
    $complaintId,
    $name,
    $email,
    $school,
    $category,
    $description
]);

    echo json_encode([
        "success" => true,
        "message" =>
            "Grievance submitted
            successfully.",
        "complaint_id" =>
            $complaintId
    ]);

    exit;
}

}

if (
    $method === "GET" &&
    isset($_GET["action"]) &&
    $_GET["action"] === "track"
) {
    $id =
        $_GET["id"] ?? "";

    $stmt =
        $pdo->prepare(

```

```
"SELECT *
FROM complaints
WHERE complaint_id = ?"

);

$stmt->execute([
    $id
]);

$complaint =
    $stmt->fetch();

if ($complaint) {

    echo json_encode([

        "success" => true,

        "data" => $complaint

    ]);

} else {

    echo json_encode([

        "success" => false,

        "message" =>
            "Complaint not found."

    ]);

}

exit;
```

```
}

http_response_code(400);

echo json_encode([

    "success" => false,

    "message" =>
        "Invalid API request."

]);

?>

Database Implementation
database/schema.sql
CREATE DATABASE IF NOT
EXISTS
grievance_portal;

USE grievance_portal;

CREATE TABLE users (

    id INT AUTO_INCREMENT
    PRIMARY KEY,

    name VARCHAR(100) NOT
    NULL,

    email VARCHAR(150)
    UNIQUE NOT NULL,

    password VARCHAR(255)
    NOT NULL,

    role ENUM(
```

```
'Student',
'Parent',
'Teacher',
'Administrator',
'Government Official'
)
DEFAULT 'Student',

created_at TIMESTAMP
DEFAULT
CURRENT_TIMESTAMP

);
CREATE TABLE complaints (

    id INT AUTO_INCREMENT
    PRIMARY KEY,

    complaint_id VARCHAR(50)
    UNIQUE NOT NULL,

    name VARCHAR(100)
    NOT NULL,

    email VARCHAR(150)
    NOT NULL,

    school VARCHAR(200)
    NOT NULL,

    category VARCHAR(100)
    NOT NULL,

    description TEXT
    NOT NULL,

    status VARCHAR(50)
    DEFAULT 'Submitted',

    created_at TIMESTAMP
```

```
DEFAULT
CURRENT_TIMESTAMP,

    updated_at TIMESTAMP
    DEFAULT
CURRENT_TIMESTAMP
ON UPDATE
CURRENT_TIMESTAMP

);

CREATE TABLE
complaint_categories (

    id INT AUTO_INCREMENT
    PRIMARY KEY,

    category_name VARCHAR(100)
    UNIQUE NOT NULL,

    description TEXT

);

CREATE TABLE complaint_status (

    id INT AUTO_INCREMENT
    PRIMARY KEY,

    complaint_id VARCHAR(50)
    NOT NULL,

    status VARCHAR(50)
    NOT NULL,

    remarks TEXT,

    updated_by INT,
```

```

        created_at TIMESTAMP
        DEFAULT
        CURRENT_TIMESTAMP

);

CREATE TABLE notifications (

    id INT AUTO_INCREMENT
    PRIMARY KEY,

    user_id INT NOT NULL,

    complaint_id VARCHAR(50),

    message TEXT NOT NULL,

    is_read BOOLEAN
    DEFAULT FALSE,

    created_at TIMESTAMP
    DEFAULT
    CURRENT_TIMESTAMP

);

CREATE TABLE schools (

    id INT AUTO_INCREMENT
    PRIMARY KEY,

    school_name VARCHAR(200)
    NOT NULL,

    address TEXT,

    district VARCHAR(100),

    state VARCHAR(100),

    created_at TIMESTAMP
    DEFAULT
    CURRENT_TIMESTAMP
);

CREATE TABLE reports (

    id INT AUTO_INCREMENT
    PRIMARY KEY,

    report_type VARCHAR(100),

    generated_by INT,

    generated_at TIMESTAMP
    DEFAULT
    CURRENT_TIMESTAMP
);

```

B.7.8 Implementation Summary

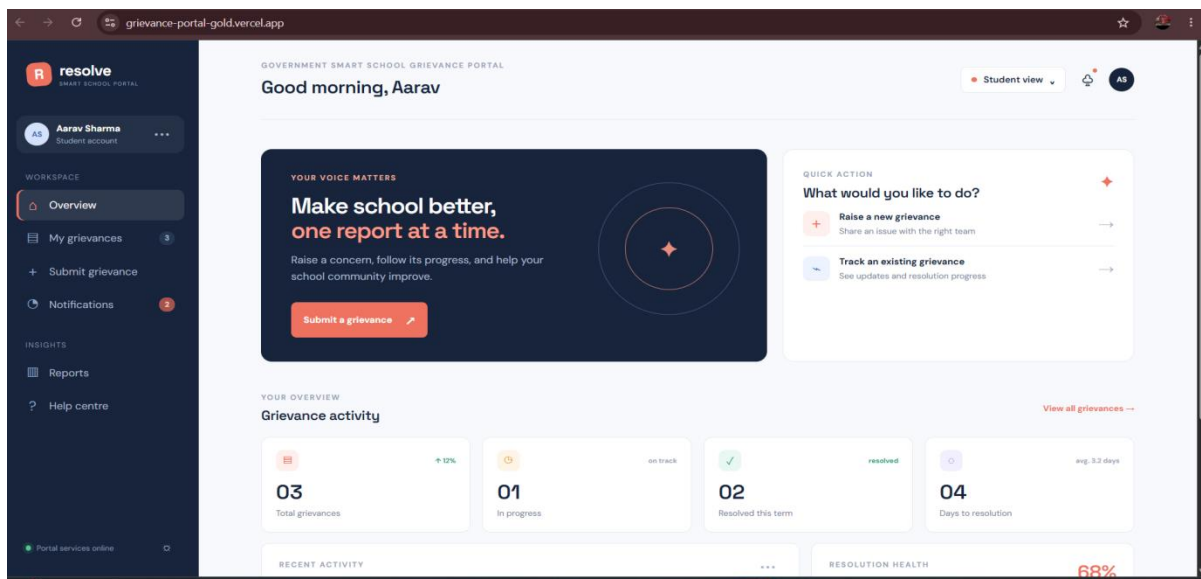
The implemented Government Smart School Grievance Portal consists of a web-based frontend, PHP backend services, and a MySQL database. The frontend provides the user interface for login, grievance submission, and grievance tracking. The backend processes requests, validates user information, generates complaint

IDs, and communicates with the database. The database stores users, complaints, categories, status information, notifications, schools, and reports.

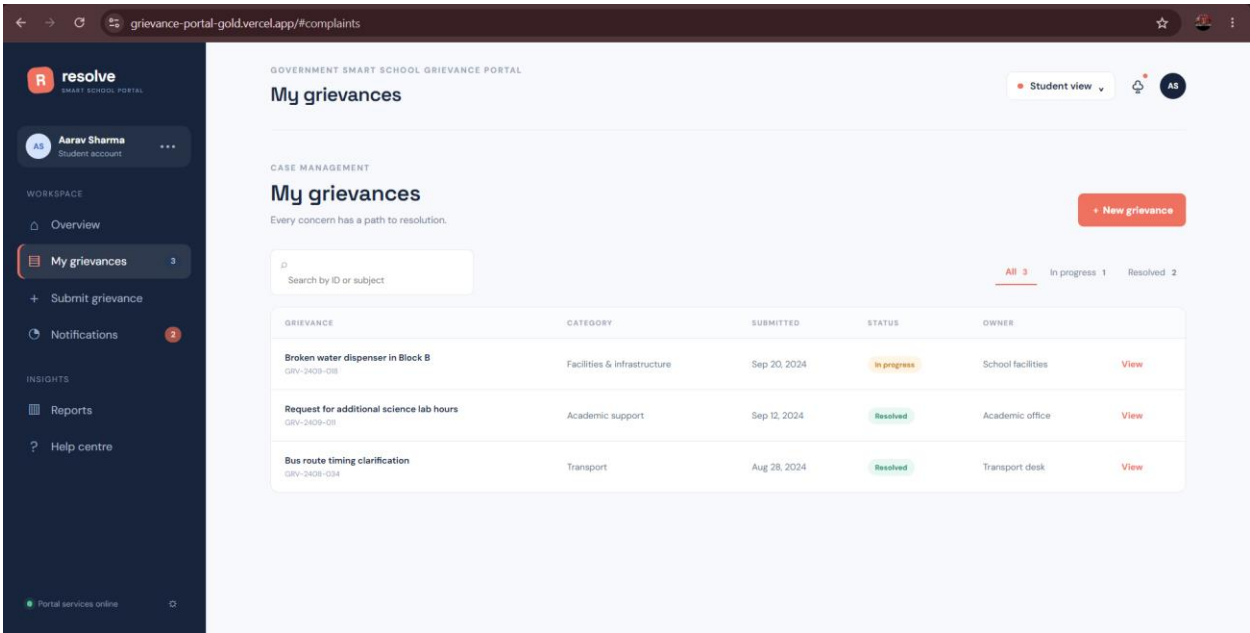
This implementation corresponds with the SRS requirement that the system provide authentication, grievance management, complaint tracking, reporting, and database support.

Implementation Screenshots

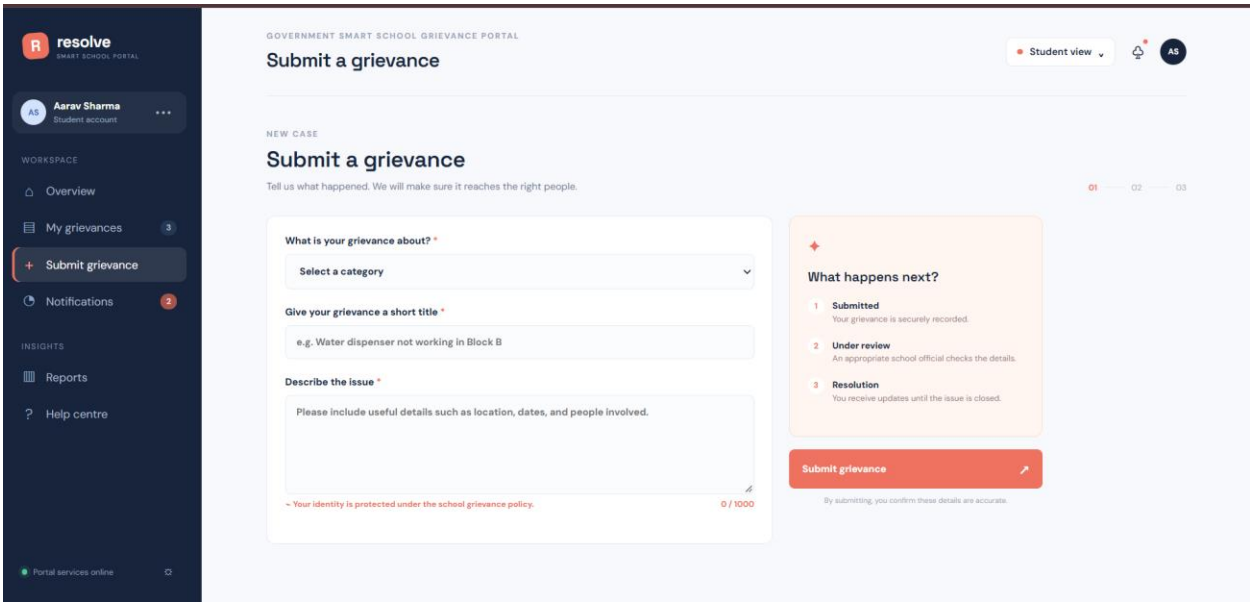
The following screenshots are generated from the supplied frontend implementation files and show the principal implemented portal views.



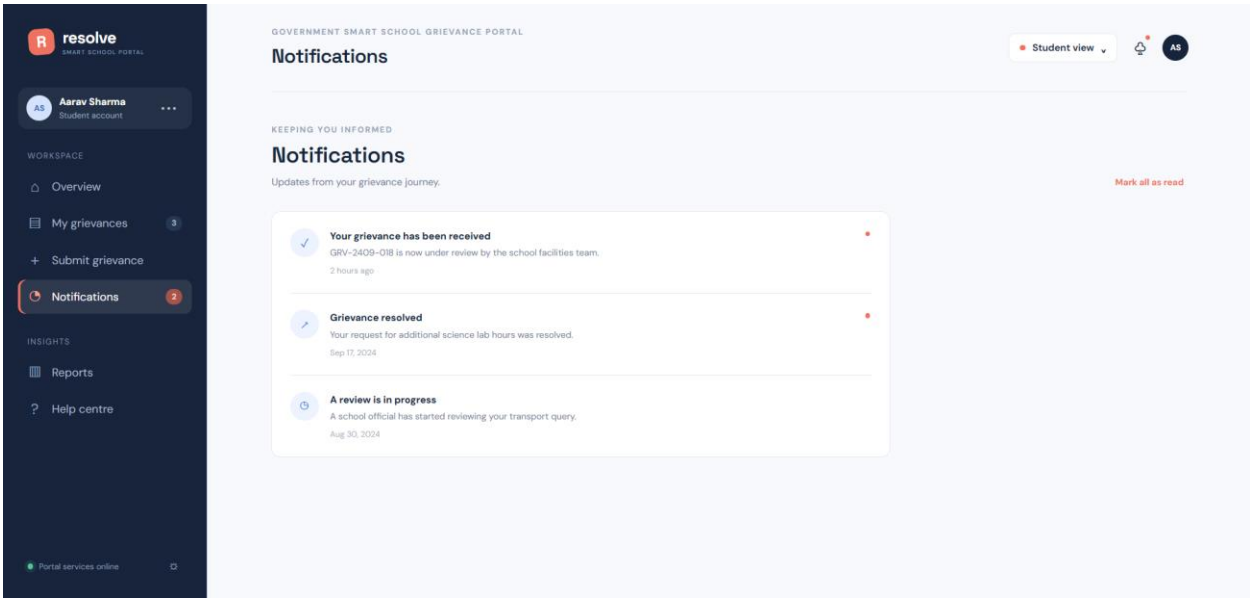
Government Smart School Grievance Portal Dashboard



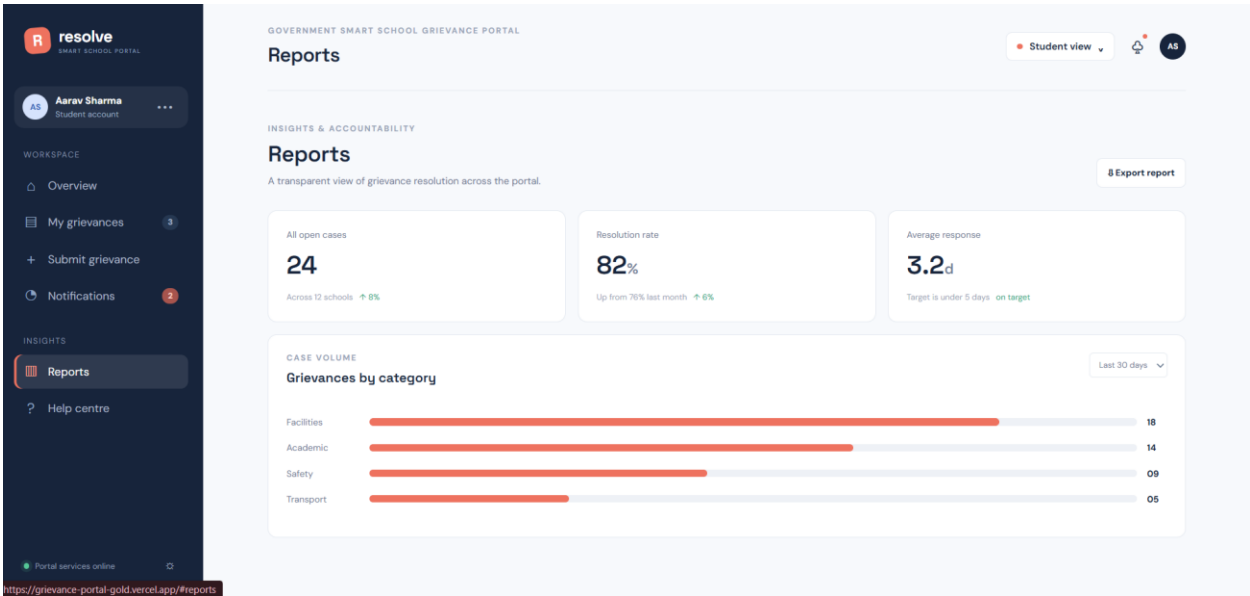
My Grievances / Case Management



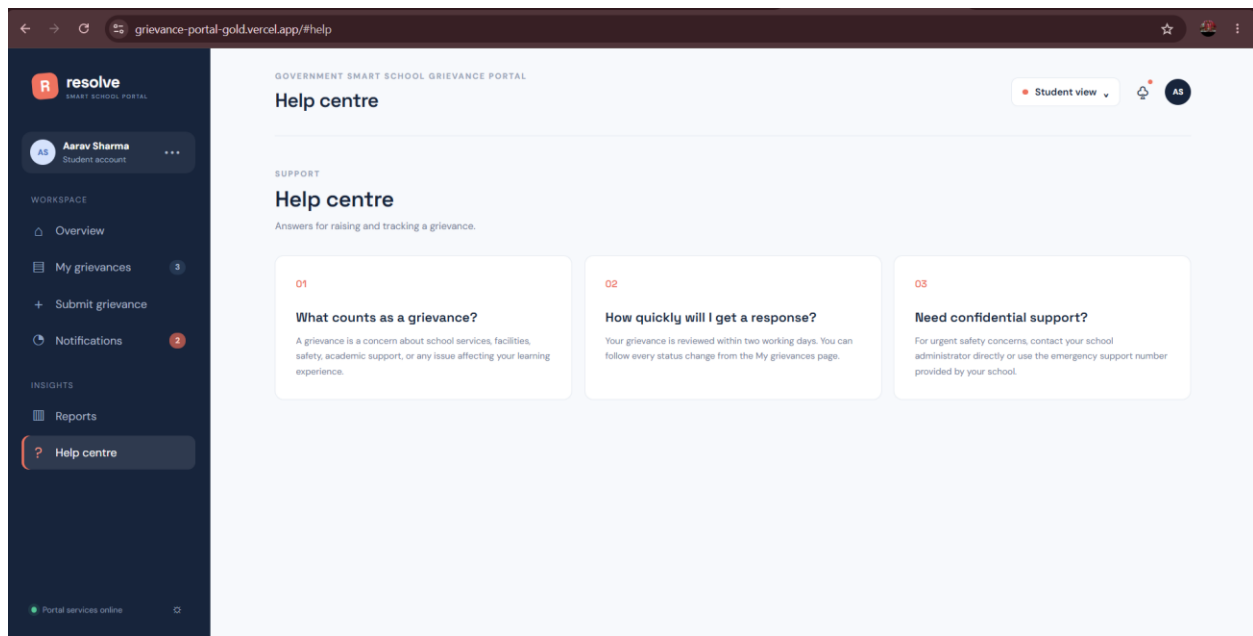
Submit a Grievance



Notifications



Reports and Analytics



Help Centre

E-Resources:

Git Hub Link: <https://github.com/lavishraina37-ai/grievance-portal..git>

